

124. Binary Tree Maximum Path Sum

Problem Summary

Given the root of a binary tree, return the maximum path sum of any non-empty path. A path can start and end at any node and doesn't need to pass through the root.

Approach

Post-order DFS — at each node compute the best path through it (left + right + root.val) and update global max. Return only the best single arm (root.val + max(left, right)) to parent since a path can't split.

Algo

1. Initialize `maxPath = Integer.MIN_VALUE`.
2. In `solve`: if `root == null` return 0.
3. `left = Math.max(0, solve(root.left))` — ignore negative contributions.
4. `right = Math.max(0, solve(root.right))`.
5. `maxPath = Math.max(maxPath, left + right + root.val)`.
6. Return `root.val + Math.max(left, right)`.

Key Insights

- `Math.max(0, solve(...))` — if subtree contributes negatively, treat it as 0 (don't include it).
- Function does two jobs: updates global max (full path) and returns single arm (for parent).
- `Integer.MIN_VALUE` not 0 as initial max — tree can have all negative values.
- Path through node = `left + right + root.val`; arm returned to parent = `root.val + max(left, right)`.

Time Complexity & Space Complexity

$O(n)$ — every node visited once.

$O(h)$ — recursion stack.

Observations

- `Integer.MIN_VALUE` as initial value handles all-negative trees correctly — 0 would give wrong answer.
- `Math.max(0, ...)` for left/right neatly handles negative subtrees — no if/else needed.
- Single arm returned to parent because a path can't fork — this is the key constraint.

Problem List

124. Binary Tree Maximum Path Sum

Solved

Hard

Topics

Companies

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at **most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the **root** of a binary tree, return the **maximum path sum** of any **non-empty path**.

Example 1:

```
graph TD; 1((1)) --- 2((2)); 1 --- 3((3))
```

Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:

```
graph TD; -10((-10)) --- -5((-5)); -10 --- -3((-3))
```

Accepted

96 / 96 testcases passed

Sahil Khan submitted at Jul 31, 2026 22:35

₹583.25

Unlock the Full LeetCode Experience

Runtime

0 ms | Beats 100.00%

Memory

46.74 MB | Beats 38.35%

Code

Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

root = [1,2,3]

Output